# OpenGuide

## 开源项目新手贡献智能向导

输入 GitHub 仓库 URL，生成**分阶段、可回溯原文**的新手贡献路线图。未通过原文核验的命令隐藏，执行与用户效果另行验证。

| | |
|---|---|
| **申报方向** | 保持方向一：开源赋能的 AI 应用创新 |
| **作品名称** | OpenGuide开源贡献向导 |
| **验证状态** | 工程验证已进行；用户效果与上游贡献闭环待补充 |
| **许可** | MIT |
| **文档版本** | 2026-10-18 · 修订工作稿 |

# 目录

1. 作品简介与选题论证
    - 1.1 问题
    - 1.2 我们的答案
    - 1.3 需求与验证边界
2. 系统设计：四段管线
    - 2.1 为什么不是「clone 全文 → LLM 写总结」
    - 2.2 M1：四视角分层索引（技术差异点核心）
    - 2.3 M2：证据锚点校验（防幻觉核心）
    - 2.4 无 Key 也能跑（演示与离线评审友好）
3. 关键工程决策
4. 开源清单与引用
5. 验证证据：四层评测
    - 5.1 五个指标（benchmark/metrics.py，规则化、确定性）
    - 5.2 结果口径与实验状态
    - 5.3 golden 仓库集
    - 5.4 自动测试
    - 5.5 第二轮命令实跑（固定提交与环境）
    - 5.6 连续流程与执行上下文
6. 真实贡献记录（方向三佐证）
7. 运行与复现
    - 7.1 一键部署（评审冷启动推荐）
    - 7.2 本地开发
    - 7.3 接口
8. 项目路线图与开源计划
    - 8.1 已交付
    - 8.2 后续计划
    - 8.3 开源计划
9. 隐私、授权与 AI 辅助开发

附：文档索引

> 当前状态：见 国赛完善与验收记录。本文件与 PDF 为修订工作稿；真实用户效果与贡献证据尚待补齐。

**作品名称**：OpenGuide开源贡献向导 **申报方向**：保持现有方向一（开源赋能的 AI 应用创新）；申报分类如需调整，先由团队确认。项目服务于开源新手贡献场景。 **一句话定位**：输入一个 GitHub 仓库 URL，输出一份**带证据、可执行、会陪你走完每一步**的新手贡献路线图。

# 1. 作品简介与选题论证

## 1.1 问题

开源生态最大的摩擦不在"写代码"，而在**第一次贡献之前的那段路**：环境怎么搭、测试怎么跑、 第一个 issue 怎么选、PR 怎么提。这些信息散落在 README、CONTRIBUTING、docs/、Makefile、 CI workflow 与 issue 标签里，新手既不知道去哪找，也不知道找得对不对。

通用聊天机器人能给一段"看起来像那么回事"的答案，但它**无法保证答案来自这个仓库**—— 编造的安装命令、不存在的 issue 编号，正是新手最容易被坑的地方。

## 1.2 我们的答案

OpenGuide 把非结构化仓库加工成**结构化、可检索、带证据、可分步执行的操作手册**， 并守住一条硬边界：

> **无证据不宣称。** 每条建议都要能回溯到仓库里的真实文件或 issue；不能确认的， 宁可标注 `missing` 并建议提问，也不编造安装命令或 issue 编号。

## 1.3 需求与验证边界

目标用户是有基础编程经验、尚未完成首次贡献的学生。项目解决查找环境与测试说明、识别任务、按步骤操作的问题。 目前尚缺真实用户实验与可核验的上游贡献闭环，具体补充方案见 `user_study_protocol.md`。 不从功能实现或测试通过直接推断学习效果。

## 2. 系统设计：四段管线

```
GitHub 仓库 URL
      │
      ▼
┌─────────────────────────────────────────────────────┐
│ M0 仓库勘探与画像        backend/src/backend/core/github │
│   只抓"决策相关"信号，不 clone 全文                   │
│   → 语言占比 / 构建面 / good-first-issue / 活跃度 / 协议 │
│   → 磁盘缓存（规避限流）                              │
└─────────────────────────────────────────────────────┘
      │  Profile（固定 schema）
      ▼
┌─────────────────────────────────────────────────────┐
│ M1 分层视角化索引        backend/src/backend/core/index  │
│   按"新手阅读路径"把仓库重组为四个视角，各自独立建索引  │
│   V1 入口与约定 / V2 架构地图 / V3 可执行路径 / V4 issue │
└─────────────────────────────────────────────────────┘
      │  四视角索引
      ▼
┌─────────────────────────────────────────────────────┐
│ M2 结构化指南生成        backend/src/backend/core/generate│
│   画像驱动决策树 → Stage A-D 分阶段检索 → schema 约束生成│
│   → 证据锚点逐条校验（不通过就降级为 missing）         │
└─────────────────────────────────────────────────────┘
      │  Guide（每步带 evidence）
      ▼
┌─────────────────────────────────────────────────────┐
│ M3 分步引导交互          frontend/src/components/GuideWizard│
│   向导卡片流 + 三步追问 + 常驻证据面板                 │
└─────────────────────────────────────────────────────┘
```

### 2.1 为什么不是「clone 全文 → LLM 写总结」

现有流程面临的三个问题，正是四段管线的设计依据：

| 失配 | 说明 | 对策 |
|---|---|---|
| 规模失配 | 数千文件塞不进上下文，噪声淹没信号 | M0 只抓决策相关信号；M1 分视角建索引 |
| 信息类型失配 | "如何搭环境"分散在 README / CONTRIBUTING / docs / workflow | V3 把可执行路径**跨文件聚合**到一起 |
| 能力前提失配 | 新手卡住时不会定位问题 | 指南带**预期结果**与**失败分支**；三步追问兜底 |

### 2.2 M1：四视角分层索引（技术差异点核心）

通用代码分块未必能直接回答上手任务；本项目检验按任务路由检索是否有助于找到相关说明。 我们按新手阅读路径重组为四个视角，各视角独立建索引：

| 视角 | 内容来源 | 回答什么 |
|---|---|---|
| V1 入口与约定 | README + CONTRIBUTING + 规范 + license | 贡献流程、协议边界 |
| V2 架构地图 | 目录树 + 顶层结构 | 代码怎么组织、改哪在哪 |
| V3 可执行路径 | 构建清单 + Makefile/脚本 + 安装章节 | **怎么跑起来 / 跑测试** |
| V4 issue 槽位 | good-first-issue 标题/正文/标签 | 现在能做什么 |

**V3 的准入规则值得单说**：一个文档只有**真的含命令行**（或本身就是构建文件）才进 V3。 这条规则是被真实缺陷逼出来的——`psf/requests` 的 `docs/dev/contributing.rst` 是纯叙述文档 （一条命令都没有）， 早期版本仅凭文件名含 `contribut` 就把它收进 V3，它靠 11 个分块把 真正含 `python -m pytest tests` 的 `Makefile` 挤出了检索结果，导致"怎么跑测试"答不出来。 同类被排除的还有 changelog / history / authors——它们是"变更与署名"，不是可执行路径。

### 2.3 M2：证据锚点校验（防幻觉核心）

生成不是"让模型自由发挥"，而是**先检索、再填空、最后审计**：

1. 每个 Stage 只拿到该阶段检索到的仓库原文片段；
2. 模型按严格 JSON schema 填写 `GuideStep`（title / command / expected / fail_hints / evidence）；
3. **证据审计**：分别验证来源路径、行号、引用原文和命令原文；

无法核验则标记 `missing`；未通过原文核验的命令隐藏。原文匹配不能证明语义正确或实际执行成功；

1. 仍缺出处时定向检索，只有引用或命令能在候选原文核验时才回填；不能将无关文件作为证据。

### 2.4 无 Key 也能跑（演示与离线评审友好）

未配置大模型时，生成走**确定性启发式**路径：用规则从检索到的原文里挑出**属于该阶段** 的真实命令（A 安装 / B 测试 / C issue / D git·PR）， 挑不到就留空。 这条路让 CI 与离线评审无需任何密钥即可跑通全链路。

## 3. 关键工程决策

| 决策 | 理由 |
|---|---|
| 不 clone 全文，只走 GitHub API | 省带宽、可缓存、能在无 git 环境运行；配合 TTL 磁盘缓存规避限流 |
| 向量检索可降级 | chromadb 不可用或嵌入模型缺失时，回退到确定性的词元重叠检索，测试与演示在离线环境仍可跑 |
| 证据来源保留仓库真实大小写 | 前端据此拼"打开原始出处"链接，大小写错了会 404 |
| 意图路由用整词匹配 | 早期用子串匹配，短词 `pr` 会命中 `improve`/`progress`，把问题错误路由到"贡献流程" |
| LLM 配置只存会话内存 | Key 不落盘、不进日志；页面右上角可临时填写 |

## 4. 开源清单与引用

本项目以 **MIT** 许可发布（见 `LICENSE`）。**不自称纯开源**：编排、分层索引、证据校验、 评测体系为团队自主实现；通用大模型走商业 API。

完整的第三方组件、开源模型与商业服务清单及其许可条件，逐项列于 `third_party_disclosure.md`。

## 5. 验证证据：四层评测

评测分四层，**口径红线**：仓库内样例/烟测**不得**宣称生成质量；任何质量数字必须能回溯到 **人工复核过的 golden 仓库集**。详见 `../benchmark/README.md`。

| 层 | 命令 | 作用 | 是否产出质量数字 |
|---|---|---|---|
| **L1 烟测** | `python -m benchmark.l1_smoke` | 离线验证 URL→画像→决策树→索引→生成→schema→证据 全链路通 | **否**（明确不产出） |
| **L2 黄金评估** | `python -m benchmark.l2_golden.runner` | 对 golden 仓库跑真实生成，按五维度打分 | 是（人工复核集） |
| **L3 消融** | `python -m benchmark.l3_ablation.runner` | 同批仓库对比「朴素单次生成」vs「分层四视角管线」，含耗时 | 是（同基准防漂移） |
| **L4 真实贡献** | `python -m benchmark.l4_contribution.runner` | 用本产品实际提交 PR 的记录 | 记录，非评分 |

### 5.1 五个指标（`benchmark/metrics.py`，规则化、确定性）

| 指标 | 含义 |
|---|---|
| `step_coverage` 步骤完整率 | 生成的步骤覆盖了多少 golden 步骤 |
| `evidence_hit` 证据命中率 | 生成步骤的证据来源能否解析到真实仓库文件/issue |
| `command_correct` 参考命令匹配率 | 严格匹配动作、目标和参数，仅容许明确的等价形式 |
| `command_exec` 命令可执行率 | 命令能否在真实 clone 上执行（"跑得动吗"） |
| `assert_rate` 非缺失来源比例 | 非 `missing` 步骤占比，不表示事实正确性 |

另记录 **单仓库生成耗时**（`elapsed_s`），用于回答"分层管线多换来了什么、多花了多少时间"。

### 5.2 结果口径与实验状态

2026-10-06 的旧 L2 结果来自规则生成；旧 L3 使用人工构造的 fake 基线，只能演示错误检测，不能证明真实模型效果。 命令匹配和步骤覆盖定义已修复，旧结果不能与新指标直接比较。原始数据保留在 `benchmark/reports/`，不得作为当前版本质量结论。

新增 `python -m benchmark.controlled_ablation`，使用真实模型、固定源码快照、四组对照、原始输出、失败与 token 用量记录。 证据审计组复用原始生成，避免重新采样混淆。当前结果仅作探索性分析，golden 还需按快照 SHA 人工复核，命令执行与用户效果另行验证。 本轮完成五仓库探索性实验，63 次成功模型调用，未测命令执行和用户效果；四视角并未证明全面优于普通 RAG。详见 `real_model_findings_20261018.md`、`competition_readiness.md` 和 `evaluation_report.md`。

### 5.3 golden 仓库集

旧标注记录中 5 个仓库标记为 `verified: true`；本轮尚需按固定 SHA 重新人工复核。以下为原集合（见 `benchmark/golden/repos.json`）：

| 仓库 | 类型 | 覆盖特征 |
|---|---|---|
| `psf/requests` | Python 库 | 文档最全、最典型的新手入口 |
| `pallets/click` | Python CLI 框架 | pyproject + pytest，构建面干净 |
| `cookiecutter/cookiecutter` | 项目脚手架 | 脚手架场景代表 |
| `mochajs/mocha` | JavaScript 测试框架 | **跨语言**（package.json + npm test） |
| `camelot-dev/camelot` | PDF 表格抽取 | 依赖较重，验证鲁棒性 |

### 5.4 自动测试

| 套件 | 命令 | 数量 |
|---|---|---|
| 后端单测/集成 | `python -m pytest backend/tests -q` | **以当前测试运行记录为准** |
| 评测体系单测 | `python -m pytest benchmark/tests -q` | **以当前测试运行记录为准** |
| 合计 | — | **以当前测试运行记录为准** |
| CI | `.github/workflows/ci.yml` | 后端测试 + 前端构建 + 密钥扫描 |

CI 全程走 hermetic 路径（`USE_CHROMA=false`），**不下载任何嵌入模型**，秒级完成。

### 5.5 第二轮命令实跑（固定提交与环境）

Requests 开发依赖安装通过；完整 CI 为 625 passed、6 failed、3 skipped、1 xfailed，整体失败。另选的数据结构子集 24 passed，不能替代全量结果。 Click 直接 pytest 基线为 2179 passed、107 skipped、31000 deselected、1 xfailed，不代表 tox random 或 stress 环境通过。 Requests 的 Windows 符号链接问题经定位，证书单例复测通过；完整 CI 没有在链接修复后重跑，其他网络/代理相关失败仍待解决。

本轮据此增加简单 make 目标原文展开、保存指南执行记录、抓取缺失状态和符号链接提示。执行计划、原始模型回放与实际执行结果分开记录，不将它们混成一个准确率。 详见 `execution_findings_20261018.md` 与 `benchmark/reports/execution_validation_20261018.json`。

### 5.6 连续流程与执行上下文

克隆仓库已前移到安装之前，步骤重新编号并标出流程前置关系；Makefile 命令显示按构建文件推导的执行目录和所需输入路径，资料不足时明确待确认。基础完成检查与仓库原文预期分开展示。 历史执行记录除命令和提交外，还核对已记录的执行目录，不能挪用其他目录的结果。 本轮仅对既有输出作确定性回放，没有新增模型抽样或完整测试结论，详见 `workflow_round3_20261018.md`。

## 6. 真实贡献记录（方向三佐证）

L4 记录"用本产品自己走通一个真实 PR"的闭环证据，落盘于 `benchmark/l4_contribution/records.json`，用一行代码补录：

```
from benchmark.l4_contribution.runner import add_record
add_record("owner", "repo", "pr", "https://github.com/owner/repo/pull/123", "用 OpenGuide 走通的文档改进")
```

状态语义（避免"待补充"被误读成"失败"）：

- `PENDING`：尚无记录（退出码 0，明确标注待补充）
- `OK`：已有记录，逐条打印
- `FAIL`：记录文件损坏（真错误）

> **口径**：没有记录就不会被算作"闭环已完成"——报告明确标 `PENDING`，评审一眼可见此项待补。

## 7. 运行与复现

### 7.1 一键部署（评审冷启动推荐）

```
cp .env.example .env          # 可选：填 GITHUB_TOKEN / LLM_API_KEY；不填也能跑
docker compose up --build
# 浏览器打开 http://127.0.0.1:5173
```

前端由 nginx 提供静态站点并把 `/api` 反向代理到后端容器（同源，免 CORS）。 默认 `USE_CHROMA=false` 走确定性检索，**零模型下载、秒级启动**，适合离线评审。

### 7.2 本地开发

```
# 后端
cd backend && uv sync && uv run uvicorn backend.api.main:app --port 8000
# 前端
cd frontend && pnpm install && pnpm dev     # http://127.0.0.1:5173
```

### 7.3 接口

| 方法 | 路径 | 说明 |
|---|---|---|
| POST | `/api/profile` | `{url}` → 仓库画像 |
| POST | `/api/search` | `{url, query, kind?}` → 按视角路由的检索结果 |
| POST | `/api/guide` | `{url, use_llm}` → Stage A–D 结构化指南 |
| POST | `/api/followup` | 三步追问：这步太粗 / 看不懂为什么 / 报错了 |
| GET/POST | `/api/llm-config` | 会话内 LLM 配置（不回显 Key） |
| GET | `/health` | 存活检查 |

# 8. 项目路线图与开源计划

## 8.1 已交付

- 四段管线（M0 勘探 / M1 分层索引 / M2 结构化生成 / M3 分步引导）全部打通
- 四层评测体系（L1–L4）+ 5 个已人工复核的 golden 仓库
- Docker 一键部署、演示脚本、第三方许可披露、变更记录

## 8.2 后续计划

| 方向 | 内容 |
|---|---|
| 评测扩容 | golden 集从 5 个扩到 15–20 个，覆盖更多语言与构建系统（Rust / Go / Java） |
| 开源模型对比 | 用同一评估集对比开源模型与商业模型的输出质量（点题"AI + 开源"，不做过度承诺） |
| V2 架构视角升级 | 引入 tree-sitter 符号表，把"改哪个文件"从目录级细化到符号级 |
| 追问增强 | 报错定位接入更多语言生态的规则库 |
| 真实贡献 | 持续用本产品向小仓库提交 PR，滚动更新 L4 记录 |

## 8.3 开源计划

本作品以 MIT 许可开源。仓库目前已公开，接受外部 issue 与 PR—— 用自己做的工具来管理自己的新手贡献者，是这个项目最好的验证方式。

# 9. 隐私、授权与 AI 辅助开发

见 `privacy_and_license.md` 与 `ai_assistance_disclosure.md`。

# 附：文档索引

| 文档 | 内容 |
|---|---|
| `task_book.md` | 任务书（选题论证 / 技术方案 / 里程碑） |
| `demo_script.md` | 现场演示脚本（3 个真实仓库 + 兜底方案） |
| `video_script.md` | 演示视频分镜脚本 |
| `code_outline.md` | 代码大纲与接口约定 |
| `third_party_disclosure.md` | 第三方资源与许可披露 |
| `privacy_and_license.md` | 隐私与授权说明 |
| `cold_start_checklist.md` | 提交包冷启动演练记录 |
| `evaluation_report.md` | 评测报告（L2/L3 实测数字与归因） |
| `../output/pdf/OpenGuide-    .pdf` | 本说明的 PDF 版本（提交用） |
| `../tools/render_submission_pdf.py` | PDF 生成脚本（改完 Markdown 后重新生成） |
| `changelog.md` | 变更记录（含修复根因与验证口径） |
| `../benchmark/README.md` | 评测方法论与口径红线 |